Name: A.HemaCharith

Reg.no: 23bce7737

1. Use MongoDB to implement the following DB operations

   1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

```
Connection=true&serverSelectionTimeoutMS=2000
/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-11T14:56:46.410+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use("vehicles")
switched to db vehicles
```

   2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin      40.00 KiB
config    100.00 KiB
local      40.00 KiB
vehicles   16.00 KiB
```

   3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
vehicles> db.createCollection("two_wheelers",{capped:true,size:100000}
| db.createCollection("four_wheelers")
{ ok: 1 }
```

   4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Honda Shine",
| model: "gear",
| category: "125cc",
| colors_available: ["black","red","blue"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2021-05-10"),
| price: 85000
| },
|
| {
| bike_name: "TVS Apache",
| model: "gear",
| category: "150cc",
| colors_available: ["black","sport red"],
| manufacturer: "TVS",
| performance: 9,
| timestamp: new Date("2022-03-15"),
| price: 110000
| },
|
| {
| bike_name: "Activa 6G",
| model: "gearless",
| category: "125cc",
| colors_available: ["red","blue","white"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2020-07-20"),
| price: 78000
| },
|
```

```
| {
| bike_name: "Bajaj Pulsar",
| model: "gear",
| category: "200cc",
| colors_available: ["black","blue"],
| manufacturer: "Bajaj",
| performance: 9,
| timestamp: new Date("2023-02-11"),
| price: 140000
| },
|
| {
| bike_name: "Hero Splendor",
| model: "gear",
| category: "100cc",
| colors_available: ["black","red"],
| manufacturer: "Hero",
| performance: 7,
| timestamp: new Date("2019-06-25"),
| price: 70000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b190c32606419eff7c2907'),
    '1': ObjectId('69b190c32606419eff7c2908'),
    '2': ObjectId('69b190c32606419eff7c2909'),
    '3': ObjectId('69b190c32606419eff7c290a'),
    '4': ObjectId('69b190c32606419eff7c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Maruti Swift",
| model: "own",
| category: "car",
| variants: ["vxi","zxi","petrol"],
| manufacturer: "Maruti",
| performance: 8,
| timestamp: new Date("2022-01-10"),
| price: 650000
| },
|
| {
| vehicle_name: "Tata Nexon",
| model: "own",
| category: "car",
| variants: ["diesel","petrol"],
| manufacturer: "Tata",
| performance: 9,
| timestamp: new Date("2023-04-15"),
| price: 900000
| },
|
| {
| vehicle_name: "Ashok Leyland Truck",
| model: "commercial",
| category: "heavy truck",
| variants: ["diesel"],
| manufacturer: "Ashok Leyland",
| performance: 7,
| timestamp: new Date("2021-09-20"),
| price: 2000000
| },
|
```

```
| {
| vehicle_name: "Mahindra Bolero",
| model: "own",
| category: "car",
| variants: ["diesel"],
| manufacturer: "Mahindra",
| performance: 8,
| timestamp: new Date("2020-12-01"),
| price: 950000
| },
|
| {
| vehicle_name: "Eicher Bus",
| model: "commercial",
| category: "bus",
| variants: ["diesel"],
| manufacturer: "Eicher",
| performance: 7,
| timestamp: new Date("2019-08-18"),
| price: 2500000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b190cb2606419eff7c290c'),
    '1': ObjectId('69b190cb2606419eff7c290d'),
    '2': ObjectId('69b190cb2606419eff7c290e'),
    '3': ObjectId('69b190cb2606419eff7c290f'),
    '4': ObjectId('69b190cb2606419eff7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b190c32606419eff7c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'black', 'red', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2021-05-10T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b190c32606419eff7c2908'),
    bike_name: 'TVS Apache',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'sport red' ],
    manufacturer: 'TVS',
    performance: 9,
    timestamp: ISODate('2022-03-15T00:00:00.000Z'),
    price: 110000
  },
  {
    _id: ObjectId('69b190c32606419eff7c2909'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'blue', 'white' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2020-07-20T00:00:00.000Z'),
    price: 78000
  },
```

```
  {
    _id: ObjectId('69b190c32606419eff7c290a'),
    bike_name: 'Bajaj Pulsar',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 9,
    timestamp: ISODate('2023-02-11T00:00:00.000Z'),
    price: 140000
  },
  {
    _id: ObjectId('69b190c32606419eff7c290b'),
    bike_name: 'Hero Splendor',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'black', 'red' ],
    manufacturer: 'Hero',
    performance: 7,
    timestamp: ISODate('2019-06-25T00:00:00.000Z'),
    price: 70000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b190cb2606419eff7c290c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    price: 650000
  },
  {
    _id: ObjectId('69b190cb2606419eff7c290d'),
    vehicle_name: 'Tata Nexon',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'petrol' ],
    manufacturer: 'Tata',
    performance: 9,
    timestamp: ISODate('2023-04-15T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b190cb2606419eff7c290e'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2021-09-20T00:00:00.000Z'),
    price: 2000000
  },
```

```
  {
    _id: ObjectId('69b190cb2606419eff7c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2020-12-01T00:00:00.000Z'),
    price: 950000
  },
  {
    _id: ObjectId('69b190cb2606419eff7c2910'),
    vehicle_name: 'Eicher Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 7,
    timestamp: ISODate('2019-08-18T00:00:00.000Z'),
    price: 2500000
  }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Honda Shine', price: 85000 },
  { bike_name: 'TVS Apache', price: 110000 },
  { bike_name: 'Activa 6G', price: 78000 },
  { bike_name: 'Bajaj Pulsar', price: 140000 },
  { bike_name: 'Hero Splendor', price: 70000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Maruti Swift', price: 650000 },
  { vehicle_name: 'Tata Nexon', price: 900000 },
  { vehicle_name: 'Ashok Leyland Truck', price: 2000000 },
  { vehicle_name: 'Mahindra Bolero', price: 950000 },
  { vehicle_name: 'Eicher Bus', price: 2500000 }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b190cb2606419eff7c290d'),
    vehicle_name: 'Tata Nexon',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'petrol' ],
    manufacturer: 'Tata',
    performance: 9,
    timestamp: ISODate('2023-04-15T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b190cb2606419eff7c290e'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2021-09-20T00:00:00.000Z'),
    price: 2000000
  },
  {
    _id: ObjectId('69b190cb2606419eff7c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2020-12-01T00:00:00.000Z'),
    price: 950000
  },
```

```
    {
      _id: ObjectId('69b190cb2606419eff7c2910'),
      vehicle_name: 'Eicher Bus',
      model: 'commercial',
      category: 'bus',
      variants: [ 'diesel' ],
      manufacturer: 'Eicher',
      performance: 7,
      timestamp: ISODate('2019-08-18T00:00:00.000Z'),
      price: 2500000
    }
  ]
  vehicles> db.two_wheelers.find(
  | {performance:{$gt:5}},
  | {bike_name:1, category:1, manufacturer:1, _id:0}
  | )
  [
    { bike_name: 'Honda Shine', category: '125cc', manufacturer: 'Honda' },
    { bike_name: 'TVS Apache', category: '150cc', manufacturer: 'TVS' },
    { bike_name: 'Activa 6G', category: '125cc', manufacturer: 'Honda' },
    { bike_name: 'Bajaj Pulsar', category: '200cc', manufacturer: 'Bajaj' },
    { bike_name: 'Hero Splendor', category: '100cc', manufacturer: 'Hero' }
  ]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.four_wheelers.find(
| {performance:{$gt:5}},
| {vehicle_name:1, category:1, manufacturer:1, _id:0}
| )
[
  {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  },
  { vehicle_name: 'Tata Nexon', category: 'car', manufacturer: 'Tata' },
  {
    vehicle_name: 'Ashok Leyland Truck',
    category: 'heavy truck',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Mahindra Bolero',
    category: 'car',
    manufacturer: 'Mahindra'
  },
  { vehicle_name: 'Eicher Bus', category: 'bus', manufacturer: 'Eicher' }
]
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called 'animal' and *write* a MongoDB query to select database as 'animal'.

```
vehicles> use animal
switched to db animal
animal>
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
admin      40.00 KiB
config     96.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

```
animal> db.createCollection("wild_animals",capped:
true,size:100000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal>
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
| {
| animal_name:"Lion",
| nature:"harm",
| favorite_foods:["meat","deer","buffalo"],
| care_taker_name:"Ramesh",
| life_span:14,
| timestamp:new Date("2022-03-10"),
| expenses:50000
| },
|
| {
| animal_name:"Tiger",
| nature:"harm",
| favorite_foods:["meat","rabbit"],
| care_taker_name:"Suresh",
| life_span:15,
| timestamp:new Date("2021-06-15"),
| expenses:60000
| },
|
| {
| animal_name:"Elephant",
| nature:"harmless",
| favorite_foods:["grass","banana","sugarcane"],
| care_taker_name:"Mahesh",
| life_span:60,
| timestamp:new Date("2020-01-05"),
| expenses:80000
| },
|
| {
| animal_name:"Leopard",
| nature:"harm",
| favorite_foods:["meat","rabbit"],
| care_taker_name:"Ramesh",
| life_span:12,
| timestamp:new Date("2023-07-20"),
| expenses:45000
| },
```

```
|
| {
| animal_name:"Bear",
| nature:"harm",
| favorite_foods:["fish","honey","meat"],
| care_taker_name:"Kiran",
| life_span:20,
| timestamp:new Date("2019-09-12"),
| expenses:55000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b196192606419eff7c2911'),
    '1': ObjectId('69b196192606419eff7c2912'),
    '2': ObjectId('69b196192606419eff7c2913'),
    '3': ObjectId('69b196192606419eff7c2914'),
    '4': ObjectId('69b196192606419eff7c2915')
  }
}
animal>
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name:"Cow",
| gender:"female",
| favorite_foods:["grass","hay"],
| animal_petname:"Gauri",
| life_span:20,
| timestamp:new Date("2021-05-15"),
| expenses:20000
| },
|
| {
| animal_name:"Dog",
| gender:"male",
| favorite_foods:["meat","bones"],
| animal_petname:"Tommy",
| life_span:13,
| timestamp:new Date("2022-02-10"),
| expenses:15000
| },
|
| {
| animal_name:"Cat",
| gender:"female",
| favorite_foods:["fish","milk"],
| animal_petname:"Kitty",
| life_span:15,
| timestamp:new Date("2023-01-08"),
| expenses:10000
| },
|
| {
| animal_name:"Goat",
| gender:"female",
| favorite_foods:["grass","leaves"],
| animal_petname:"Chinni",
| life_span:12,
| timestamp:new Date("2020-11-20"),
| expenses:8000
| },
```

```
| {
| animal_name:"Horse",
| gender:"male",
| favorite_foods:["grass","hay"],
| animal_petname:"Raja",
| life_span:25,
| timestamp:new Date("2019-08-14"),
| expenses:30000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b196712606419eff7c2916'),
    '1': ObjectId('69b196712606419eff7c2917'),
    '2': ObjectId('69b196712606419eff7c2918'),
    '3': ObjectId('69b196712606419eff7c2919'),
    '4': ObjectId('69b196712606419eff7c291a')
  }
}
animal>
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b196192606419eff7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer', 'buffalo' ],
    care_taker_name: 'Ramesh',
    life_span: 14,
    timestamp: ISODate('2022-03-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b196192606419eff7c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Suresh',
    life_span: 15,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b196192606419eff7c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'banana', 'sugarcane' ],
    care_taker_name: 'Mahesh',
    life_span: 60,
    timestamp: ISODate('2020-01-05T00:00:00.000Z'),
    expenses: 80000
  },
```

```
{
  _id: ObjectId('69b196192606419eff7c2914'),
  animal_name: 'Leopard',
  nature: 'harm',
  favorite_foods: [ 'meat', 'rabbit' ],
  care_taker_name: 'Ramesh',
  life_span: 12,
  timestamp: ISODate('2023-07-20T00:00:00.000Z')

  expenses: 45000
},
{
  _id: ObjectId('69b196192606419eff7c2915'),
  animal_name: 'Bear',
  nature: 'harm',
  favorite_foods: [ 'fish', 'honey', 'meat' ],
  care_taker_name: 'Kiran',
  life_span: 20,
  timestamp: ISODate('2019-09-12T00:00:00.000Z')

  expenses: 55000
}
```

```
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b196712606419eff7c2916'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'hay' ],
    animal_petname: 'Gauri',
    life_span: 20,
    timestamp: ISODate('2021-05-15T00:00:00.000Z'),
    expenses: 20000
    gender: 'male',
    favorite_foods: [ 'meat', 'bones' ],
    animal_petname: 'Tommy',
    life_span: 13,
    timestamp: ISODate('2022-02-10T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69b196712606419eff7c2918'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: ISODate('2023-01-08T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69b196712606419eff7c2919'),
    animal_name: 'Goat',
    gender: 'female',
    favorite_foods: [ 'grass', 'leaves' ],
    animal_petname: 'Chinni',
    life_span: 12,
    timestamp: ISODate('2020-11-20T00:00:00.000Z'),
    expenses: 8000
  },
```

```
  {
    _id: ObjectId('69b196712606419eff7c291a'),
    animal_name: 'Horse',
    gender: 'male',
    favorite_foods: [ 'grass', 'hay' ],
    animal_petname: 'Raja',
    life_span: 25,
    timestamp: ISODate('2019-08-14T00:00:00.000Z'),
    expenses: 30000
  }
]
animal>
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.wild_animals.find({}, {animal_name:1, e
expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 80000 },
  { animal_name: 'Leopard', expenses: 45000 },
  { animal_name: 'Bear', expenses: 55000 }
]
animal> db.domestic_animals.find({}, {animal_name:
1, expenses:1, _id:0})
[
  { animal_name: 'Cow', expenses: 20000 },
  { animal_name: 'Dog', expenses: 15000 },
  { animal_name: 'Cat', expenses: 10000 },
  { animal_name: 'Goat', expenses: 8000 },
  { animal_name: 'Horse', expenses: 30000 }
]
animal>
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:15})
[
  {
    _id: ObjectId('69b196712606419eff7c2918'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 15,
    timestamp: ISODate('2023-01-08T00:00:00.000Z')
,
    expenses: 10000
  }
]
animal>
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ramesh"})
[
  {
    _id: ObjectId('69b196192606419eff7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer', 'buffalo' ],
    care_taker_name: 'Ramesh',
    life_span: 14,
    timestamp: ISODate('2022-03-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b196192606419eff7c2914'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Ramesh',
    life_span: 12,
    timestamp: ISODate('2023-07-20T00:00:00.000Z'),
    expenses: 45000
  }
]
animal>
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer', 'buffalo' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'banana', 'sugarcane' ],
    expenses: 80000
  },
  {
    animal_name: 'Leopard',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 45000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'honey', 'meat' ],
    expenses: 55000
  }
]
animal>
```

```
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
[
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'hay' ],
    expenses: 20000
  },
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'bones' ],
    expenses: 15000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 10000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 8000
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'hay' ],
    expenses: 30000
  }
]
animal>
```